

Near-Optimal No-Regret Algorithms for Zero-Sum Games

Constantinos Daskalakis*

Alan Deckelbaum†

Anthony Kim‡

Abstract

We propose a new no-regret learning algorithm. When used against an adversary, our algorithm achieves average regret that scales as $O\left(\frac{1}{\sqrt{T}}\right)$ with the number T of rounds. This regret bound is optimal but not rare, as there are a multitude of learning algorithms with this regret guarantee. However, when our algorithm is used by both players of a zero-sum game, their average regret scales as $O\left(\frac{\ln T}{T}\right)$, guaranteeing a near-linear rate of convergence to the value of the game. This represents an almost-quadratic improvement on the rate of convergence to the value of a game known to be achieved by any no-regret learning algorithm, and is essentially optimal as we show a lower bound of $\Omega\left(\frac{1}{T}\right)$. Moreover, the dynamics produced by our algorithm in the game setting are strongly-uncoupled in that each player is oblivious to the payoff matrix of the game and the number of strategies of the other player, has limited private storage, and is not allowed funny bit arithmetic that can trivialize the problem; instead he only observes the performance of his strategies against the actions of the other player and can use private storage to remember past played strategies and observed payoffs, or cumulative information thereof. Here, too, our rate of convergence is nearly-optimal and represents an almost-quadratic improvement over the best previously known strongly-uncoupled dynamics.

1 Introduction

Von Neumann's min-max theorem [18] lies at the origins of the fields of both algorithms and game theory. Indeed, it was the first example of a static game-theoretic solution concept: If the players of a zero-sum game arrive at a min-max pair of strategies, then no player can improve his payoff by unilaterally deviating, resulting in an equilibrium state of the game. The min-max equilibrium played a central role in von Neumann and Morgenstern's foundations of Game Theory [19],

and inspired the discovery of the Nash equilibrium [15] and the foundations of modern economic thought [14].

At the same time, the min-max theorem is tightly connected to the development of mathematical programming, as linear programming itself reduces to the computation of a min-max equilibrium, while strong linear programming duality is equivalent to the min-max theorem.¹ Given the further developments in linear programming in the past century [10, 11], we now have efficient algorithms for computing equilibria in zero-sum games, even in very large ones such as poker [6, 7].

On the other hand, the min-max equilibrium is a static notion of stability, leaving open the possibility that there are no simple distributed dynamics via which stability comes about. This turns out not to be the case, as many distributed protocols for this purpose have been discovered. One of the first protocols suggested for this purpose is *fictitious play*, whereby players switch rounds playing the pure strategy that optimizes their payoff against the historical play of their opponent (viewed as a distribution over strategies). This simple scheme, suggested by Brown in the 1950's [3], was shown to converge to a min-max equilibrium of the game by Julia Robinson [20]. However, its convergence rate has recently been shown to be exponential in the number of strategies [2]. Such poor convergence guarantees do not offer much by way of justifying the plausibility of the min-max equilibrium in a distributed setting, making the following questions rather important: *Are there efficient and natural distributed dynamics converging to the min-max equilibrium?* And *what is the optimal rate of convergence?*

The answer to the first question is, by now, very well understood. A typical source of efficient dynamics converging to min-max equilibria is online optimization. The results here are very general: If both players of a game use a no-regret learning algorithm to adapt their strategies to their opponent's strategies, then the average payoffs of the players converge to their min-max value, and their average strategies constitute an

*EECS, MIT. Email: costis@csail.mit.edu. Supported by a Sloan Foundation Fellowship and NSF CAREER Award CCF-0953960.

†Department of Mathematics, MIT. Email: deckel@mit.edu. Supported by Fannie and John Hertz Foundation, Daniel Stroock Fellowship.

‡Oracle Corporation, 500 Oracle Parkway, Redwood Shores, CA 94065. Email: tonykim@yahoo.com. Work done while the author was a student at MIT.

¹This equivalence was apparently noticed by Dantzig and von Neumann at the inception of the linear programming theory, but no rigorous account of their proof can be found in the literature. A rigorous proof of this equivalence has just been recently given by Ilan Adler [1].

approximate min-max equilibrium, with the approximation converging to 0 [4]. In particular, if a no-regret learning algorithm guarantees average regret $g(T, n, u)$, as a function of the number T of rounds, the number n of “experts”, and the magnitude u of the maximum in absolute value payoff of an expert at each round, we can readily use this algorithm in a game setting to approximate the min-max value of the game to within an additive $O(g(T, n, u))$ in T rounds, where u is now the magnitude of the maximum in absolute value payoff in the game, and n an upper bound on the players’ strategies.

For instance, if we use the *multiplicative weights update algorithm* [5, 13], we would achieve approximation $O\left(\frac{u\sqrt{\log n}}{\sqrt{T}}\right)$ to the value of the game in T rounds.

Given that the dependence of $O\left(\frac{\sqrt{\log n}}{\sqrt{T}}\right)$ in the number n of experts and the number T of rounds is optimal for the regret bound of any no-regret learning algorithm [4], the convergence rate to the value of the game achieved by the multiplicative weights update algorithm is the optimal rate that can be achieved by a black-box reduction of a regret bound to a convergence rate in a zero-sum game.

Nevertheless, a black-box reduction from the learning-with-expert-advice setting to the game-theoretic setting may be lossy in terms of approximation. Indeed, no-regret bounds apply even when a forecaster is playing against an adversary; and it may be that, when two players of a zero-sum game update their strategies following a no-regret learning algorithm, faster convergence to the min-max value of the game is possible. As concrete evidence of this possibility, take fictitious play (a.k.a. the “follow-the-leader” algorithm): against an adversary, it may be forced not to converge to zero average regret; but if both players of a zero-sum game use fictitious play, their average payoffs *do* converge to the min-max value of the game, given Robinson’s proof.

Motivated by this observation, we investigate the following: *Is there a no-regret learning algorithm that, when used by both players of a zero-sum game, converges to the min-max value of the game at a rate faster than $O\left(\frac{1}{\sqrt{T}}\right)$ with the number T of rounds?* We answer this question in the affirmative, by providing a no-regret learning algorithm, called NOREGRETEGT, with asymptotically optimal regret behavior of $O\left(\frac{u\sqrt{\log n}}{\sqrt{T}}\right)$, and convergence rate of $O\left(\frac{u \cdot \log n \cdot (\log T + (\log n)^{3/2})}{T}\right)$ to the min-max value of a game, where n is an upper bound on the number of the players’ strategies. In particular,

THEOREM 1.1. *Let $x_1, x_2, \dots, x_t, \dots$ be a sequence of*

randomized strategies over a set of experts $[n] := \{1, 2, \dots, n\}$ produced by the NOREGRETEGT algorithm under a sequence of payoffs $\ell_1, \ell_2, \dots, \ell_t, \dots \in [-u, u]^n$ observed for these experts, where ℓ_t is observed after x_t is chosen. Then for all T :

$$\frac{1}{T} \sum_{t=1}^T (x_t)^T \ell_t \geq \max_{i \in [n]} \frac{1}{T} \sum_{t=1}^T (e_i)^T \ell_t - O\left(\frac{u \cdot \sqrt{\log n}}{\sqrt{T}}\right).$$

Moreover, let $x_1, x_2, \dots, x_t, \dots$ be a sequence of randomized strategies over $[n]$ and $y_1, y_2, \dots, y_t, \dots$ a sequence of randomized strategies over $[m]$, and suppose that these sequences are produced when both players of a zero-sum game $(-A, A)$, $A \in [-u, u]^{n \times m}$, use the NOREGRETEGT algorithm to update their strategies under observation of the sequence of payoff vectors $(-Ay_t)_t$ and $(A^T x_t)_t$, respectively. Then for all T :

$$\left| \frac{1}{T} \sum_{t=1}^T (x_t)^T (-A) y_t - v \right| \leq O\left(\frac{u \cdot \log k \cdot (\log T + (\log k)^{3/2})}{T}\right),$$

where v is the row player’s value in the game and $k = \max\{m, n\}$. Moreover, for all T , the pair $\left(\frac{1}{T} \sum_{t=1}^T x_t, \frac{1}{T} \sum_{t=1}^T y_t\right)$ is an (additive) $O\left(\frac{u \cdot \log k \cdot (\log T + (\log k)^{3/2})}{T}\right)$ -approximate min-max equilibrium of the game.

Our algorithm provides the first (to the best of our knowledge) example of a *strongly-uncoupled distributed protocol* converging to the value of a zero-sum game at a rate faster than $O\left(\frac{1}{\sqrt{T}}\right)$. Strong-uncoupledness is the property of a distributed game-playing protocol under which the players can observe the payoff vectors of their own strategies at every round ($(-Ay_t)_t$ and $(A^T x_t)_t$, respectively), but:

- they do not know the payoff tables of the game, or even the number of strategies available to the other player;
- they can only use private storage to keep track of a constant number of observed payoff vectors (or cumulative payoff vectors), a constant number of mixed strategies (or possibly cumulative information thereof), and a constant number of state variables such as the round number.

The details of our model are discussed in Section 1.2. Notice that, without the assumption of strong-uncoupledness, there can be trivial solutions to the problem. Indeed, if the payoff tables of the game

were known to the players in advance, they could just privately compute their min-max strategies and use these strategies ad infinitum.² Furthermore, if the type of information they could privately store were unconstrained, they could engage in a protocol for recovering their payoff tables, followed by the computation of their min-max strategies. Even if they also didn't know each other's number of strategies, they could interleave phases in which they either recover pieces of their payoff matrices, or they compute min-max solutions of recovered square submatrices of the game until convergence to an exact equilibrium is detected. Arguably, such protocols are of limited interest in highly distributed game-playing settings.

And what is the optimal convergence rate of distributed protocols for zero-sum games? We show that, insofar as convergence of the average payoffs of the players to their corresponding values in the game is concerned, the convergence rate achieved by our protocol is essentially optimal. Namely, we show the following:³

THEOREM 1.2. *Any strongly-uncoupled distributed protocol producing sequences of strategies $(x_t)_t$ and $(y_t)_t$ for the players of a zero-sum game $(-A, A)$ such that the average payoffs of the players, $\frac{1}{T} \sum_t (x_t)^T (-A) y_t$ and $\frac{1}{T} \sum_t (x_t)^T A y_t$, converge to their corresponding value of the game, cannot do so at a convergence rate faster than an additive $\Omega(1/T)$ in the number T of rounds of the protocol. The same is true for any strongly-uncoupled distributed protocol whose average strategies converge to a min-max equilibrium.*

Future work. Our no-regret learning algorithm provides, to the best of our knowledge, the first example of a strongly-uncoupled distributed protocol converging to the min-max equilibrium of a zero-sum game at a rate faster than $\frac{1}{\sqrt{T}}$, and in fact at a nearly-optimal rate. The strong-uncoupledness arguably adds to the naturalness of our protocol, since no funny bit arithmetic, private computation of the min-max equilibrium, or anything of the similar flavor is allowed. Moreover, the strategies that the players use along the course of the dynamics are fairly natural in that they constitute

²Our notion of uncoupled dynamics is stronger than that of Hart and Mas-Colell [9]. In particular, we do not allow a player to initially have full knowledge of his utility function, since knowledge of one's own utility function in a zero-sum game reveals the entire game matrix.

³In this paper, we are concerned with bounds on *average* regret and the corresponding convergence of *average* strategy profiles. If we are concerned only with how close the *final* strategy profile is to an equilibrium, then we suspect that similar techniques to those of our paper can be used to devise a distributed protocol with fast convergence of final strategy profiles.

smoothened best responses to their opponent's previous strategies. Nevertheless, there is a certain degree of careful choreography and interleaving of these strategies, turning our protocol less simple than, say, the multiplicative weights update algorithm. So we view our contribution mostly as an *existence proof*, leaving the following as an interesting future research direction: Is there a simple variant of the multiplicative weights update protocol which, when used by the players of a zero-sum game, converges to the min-max equilibrium of the game at the optimal rate of $\frac{1}{T}$?

1.1 Learning with Expert Advice. In the learning-with-expert-advice setting, a learner has a set $[n] := \{1, \dots, n\}$ of experts to choose from at each round $t = 1, 2, \dots$. After committing to a distribution $x_t \in \Delta_n$ over the experts,⁴ a vector $\ell_t \in [-u, u]^n$ is revealed to the learner with the payoff achieved by each expert at round t . He can then update his distribution over the experts for the next round, and so forth. The goal of the learner is to minimize his average regret, measured by the following quantity at round T :

$$\max_i \frac{1}{T} \sum_{t=1}^T (e_i)^T \ell_t - \frac{1}{T} \sum_{t=1}^T (x_t)^T \ell_t,$$

where e_i is the standard unit vector along dimension i (representing the deterministic strategy of choosing the i -th expert). A learning algorithm is called *no-regret* if the regret can be bounded by a function $g(T)$ which is $o(T)$, where the function $g(T)$ may depend on the number of experts n and the maximum absolute payoff u .

The *multiplicative weights update (MWU) algorithm* is a simple no-regret learning algorithm for zero-sum games. In the MWU algorithm, a player maintains a "weight" for each pure strategy, and continually updates this weight by a multiplicative factor based on how the strategy would have performed in the most recent round. The performance of the algorithms is characterized by the following:

LEMMA 1.1. ([4]) *Let $(x_t)_t$ be the sequence of strategies generated by the MWU algorithm in view of the sequence of payoff vectors $(\ell_t)_t$ for n experts, where $\ell_t \in [-u, u]^n$. Then for all T :*

$$\max_{i \in [n]} \frac{1}{T} \sum_{t=1}^T (e_i)^T \ell_t - \frac{1}{T} \sum_{t=1}^T (x_t)^T \ell_t \leq \frac{2u}{\sqrt{2}-1} \sqrt{\frac{\ln n}{T}}.$$

⁴We use the notation Δ_n to represent the n -dimensional simplex.

1.2 Strongly-Uncoupled Dynamics. A zero-sum game is described by a pair $(-A, A)$, where A is a $n \times m$ payoff matrix, whose rows are indexed by the strategies of the “row” player and whose columns are indexed by the strategies of the “column” player. If the row player chooses a randomized, or *mixed*, strategy $x \in \Delta_n$ and the column player a mixed strategy $y \in \Delta_m$, then the row player receives payoff of $-x^T A y$, and the column player payoff of $x^T A y$. (Thus, the row player aims to minimize the quantity $x^T A y$, while the column player aims to maximize this quantity.)⁵ A *min-max* or *Nash equilibrium* of the game is then a pair of strategies (x, y) such that, for all $x' \in \Delta_n$, $x^T A y \leq (x')^T A y$, and for all $y' \in \Delta_m$, $x^T A y \geq x^T A y'$. If these conditions are satisfied to within an additive ϵ , (x, y) is called an ϵ -*approximate equilibrium*. Von Neumann showed that a min-max equilibrium exists in any zero-sum game; moreover, that there exists a value v such that, for all Nash equilibria (x, y) , $x^T A y = v$ [18]. Value v is called the *value of the column player in the game*. Similarly, $-v$ is called the *value of the row player in the game*.

We consider a repeated zero-sum game interaction between two players. At time steps $t = 1, 2, \dots$, each player chooses a mixed strategy x_t and y_t . After a player commits to a mixed strategy for that round, he observes the payoff vector $-A y_t$ and $A^T x_t$, corresponding to the payoffs achieved by each of his deterministic strategies against the strategy of the opponent. We are interested in *strongly-uncoupled efficient dynamics*, placing the following restrictions on the behavior of players:

1. **Unknown Game Matrix.** We assume that the game matrix $A \in \mathbb{R}^{n \times m}$ is unknown to both players. In particular, the row player does not know the number of pure strategies (m) available to the column player, and vice versa. (We obviously assume that the row player and column player know the values n and m of their own pure strategies.) To avoid degenerate cases in our later analysis, we will assume that both n and m are at least 2.
2. **Limited Private Storage.** The information that a player is allowed to record between rounds of the game is limited to a constant number of payoff vectors observed in the past, or cumulative information thereof, a constant number of mixed strategies played in the past, or cumulative information thereof, and a constant number of registers recording the round number and other state variables of the protocol. In particular, a player cannot record

the whole history of play and the whole history of observed payoff vectors, or use funny bit arithmetic that would allow him to keep all the history of play in one huge real number, etc.

This restriction is reminiscent of the multiplicative weights protocol, where the learner only needs to keep around the previously used mixed strategy, which he updates using the newly observed payoff vector at every round. As described in the introduction, this restriction disallows protocols where the players attempt to reconstruct the entire game matrix A , in order to privately compute a min-max equilibrium.

3. **Efficient Computations.** In each round, a player can do polynomial-time computation on his private information and the observed payoff vector.⁶

Note that the above restrictions apply only for honest players. In the case of a dishonest player (an adversary who deviates from the prescribed protocol in an attempt to gain additional payoff, for instance), we will make no assumptions about that player’s computational abilities, private storage, or private information.

A typical kind of strongly-uncoupled efficient dynamics converging to min-max equilibria can be derived by the MWU algorithm described in the previous section. In particular, if both players of a zero-sum game use the MWU algorithm to update their strategies, we can bound the average payoffs in terms of the value of the game.

COROLLARY 1.1. *Let $(x_t)_t$ and $(y_t)_t$ be sequences of mixed strategies generated by the row and column players using the MWU algorithm under observation of the sequence of payoff vectors $(-A y_t)_t$ and $(A^T x_t)_t$, respectively. Then*

$$v - C \sqrt{\frac{\ln m}{T}} \leq \frac{1}{T} \sum_{t=1}^T (x_t)^T A y_t \leq v + C \sqrt{\frac{\ln n}{T}}$$

where v is the value of the column player in the game and $C = \frac{2u}{\sqrt{2}-1}$. Moreover, for all T , $(\frac{1}{T} \sum_t x_t, \frac{1}{T} \sum_t y_t)$ is a $\left(\frac{2u}{\sqrt{2}-1} \frac{\sqrt{\ln m + \sqrt{\ln n}}}{\sqrt{T}}\right)$ -approximate Nash equilibrium of the game.

Finally, for our convenience, we make the following assumptions for all the game dynamics described in this paper. We assume that both players know a value $|A|_{max}$, which is an on the largest absolute-value

⁵Throughout this paper, if we refer to “payoff” without specifying a player, we are referring to the $x^T A y$, the value received by the column player.

⁶We will not address issues of numerical precision in this extended abstract.

payoff in the matrix A . (We assume that both the row and column player know the same value for $|A|_{max}$.) This assumption is similar to a typical bounded-payoff assumption made in the MWU protocol.⁷ We assume without loss of generality that the players know the identity of the “row” player and of the “column” player. We make this assumption to allow for protocols that are asymmetric in the order of moves of the players.⁸

1.3 Outline of Approach. Our no-regret learning algorithm is based on a gradient-descent algorithm for computing a Nash equilibrium in a zero-sum game. Our construction for converting this algorithm into a no-regret protocol has several stages as outlined below. We start with the centralized algorithm for computing Nash equilibria in zero-sum games, disentangle the algorithm into strongly-uncoupled game-dynamics, and proceed to make them robust to adversaries, obtaining our general purpose no-regret algorithm.

To provide a unified description of the game-dynamics and no-regret learning algorithms in this paper, we describe both in terms of the interaction of two players. Indeed, we can reduce the learning-with-expert advice setting to the setting where a row (or a column) player interacts with an adversarial (also called *dishonest*) column (respectively row) player in a zero-sum game, viewing the payoff vectors that the row (resp. column) player receives at every round as new columns (rows) of the payoff matrix of the game. The regret of the row (respectively column) player is the difference between the round-average payoff that he received and the best payoff he could have received against the round-average strategy of the adversary.

In more detail, our approach for designing our no-regret dynamics is the following:

- In Section 2, we present Nesterov’s Excessive Gap Technique (EGT) algorithm, a gradient-based algorithm for computing an ϵ -approximate Nash equilibrium in $O(\frac{1}{\epsilon})$ number of rounds.
- In Section 3, we “decouple” the EGT algorithm

to construct the HONESTEGTDYNAMICS protocol. This protocol has the property that, if both players honestly follow their instructions, their actions will exactly simulate the EGT algorithm.

- In Section 4.2, we modify the HONESTEGTDYNAMICS protocol to have the property that, in an honest execution, both players’ average payoffs are nearly best-possible against the opponent’s historical average strategy.
- In Section 4.3, we construct BOUNDEDEGTDYNAMICS(b), a no-regret protocol. The input b is a presumed upper bound on a game parameter (unknown by the players) which dictates the convergence rate of the EGT algorithm. If b indeed upper bounds the unknown parameter and if both players are honest, then an execution of this protocol will be the same as an honest execution of HONESTEGTDYNAMICS, and the player will detect low regret. If the player measures higher regret than expected, he detects a “failure”, which may correspond to either b not upper bounding the game parameter, or the other player significantly deviating from the protocol. However, the player is unable to distinguish what went wrong, and this creates important challenges in using this protocol as a building block for our no-regret protocol.
- In Section 4.4, we construct NOREGRETEGT, a no-regret protocol. In this protocol, the players repeatedly guess values of b and run BOUNDEDEGTDYNAMICS(b) until a player detects a failure. Every time the players need to guess a new value of b , they interlace a large number of rounds of the MWU algorithm. Note that detecting a deviating player here can be very difficult, if not impossible, given that neither player knows the details of the game (payoff matrix and dimensions) which come into the right value of b to guarantee convergence. While we cannot always detect deviations, we can still manage to obtain no-regret guarantees, via a careful design of the dynamics. The NOREGRETEGT protocol has the regret guarantees mentioned in the beginning of this introduction (see Theorem 1.1).

⁷We suspect that we can modify our protocol to work in the case where no upper bound is known, by repeatedly guessing values for $|A|_{max}$ and thereby slowing the protocol’s convergence rate by a factor polynomial in $|A|_{max}$.

⁸We can augment our protocols with initial rounds of interaction where both players select strategies at random, or according to a simple no-regret protocol such as the MWU algorithm. As soon as a round occurs with a non-zero payoff, the player who received the positive payoff designates himself the “row” player while the opponent designates himself the “column” player. Barring degenerate cases where the payoffs are always 0, we can show that this procedure is expected to terminate very quickly.

2 Nesterov’s Minimization Scheme

In this section, we introduce Nesterov’s Excessive Gap Technique (EGT) algorithm and state the necessary convergence result. The EGT algorithm is a gradient-descent approach for approximating the minimum of a convex function. In this paper, we apply the EGT algorithm to appropriate best-response functions of a zero-sum game. For a more detailed description of this

algorithm, see Appendix A. Let us define the functions $f : \Delta_n \rightarrow \mathbb{R}$ and $\phi : \Delta_m \rightarrow \mathbb{R}$ by

$$f(x) = \max_{v \in \Delta_m} x^T A v \quad \text{and} \quad \phi(y) = \min_{u \in \Delta_n} u^T A y.$$

In the above definitions, $f(x)$ is the payoff arising from the column player's best response to $x \in \Delta_n$, while $\phi(y)$ is the payoff arising from the row player's best response to $y \in \Delta_m$. Note that $f(x) \geq \phi(y)$ for all x and y , and that $f(x) - \phi(y) \leq \epsilon$ implies that (x, y) is an ϵ -approximate Nash equilibrium.

Nesterov's algorithm constructs sequences of points $x^1, x^2, \dots$ and $y^1, y^2, \dots$ such that $f(x^k) - \phi(y^k)$ becomes small, and therefore (x^k, y^k) becomes an approximate Nash equilibrium. In the EGT scheme, we will approximate f and ϕ by smooth functions, and then simulate a gradient-based optimization algorithm on these smoothed approximations. This approach for minimization of non-smooth functions was introduced by Nesterov in [17], and was further developed in [16]. Nesterov's excessive gap technique (EGT) is a gradient algorithm based on this idea. The EGT algorithm from [16] in the context of zero-sum games (see [7], [8]) is presented in its entirety in Appendix A.

The main result concerning this algorithm is the following theorem from [16]:

THEOREM 2.1. *The x^k and y^k generated by the EGT algorithm satisfy*

$$f(x^k) - \phi(y^k) \leq \frac{4\|A\|_{n,m}}{k+1} \sqrt{\frac{D_n D_m}{\sigma_n \sigma_m}}.$$

In our application of the above theorem, we will have $\|A\|_{n,m} = |A|_{\max}$ and $\frac{D_n D_m}{\sigma_n \sigma_m} = \ln n \ln m$. Our first goal is to construct a protocol such that, if both players follow the protocol, their moves simulate the EGT algorithm.

3 Honest Game Dynamics

We now use game dynamics to simulate the EGT algorithm, by "decoupling" the operations of the algorithm, obtaining the HONESTEGTDYNAMICS protocol. Basically, the players help each other perform computations necessary in the EGT algorithm by playing appropriate strategies at appropriate times. In this section, we assume that both players are "honest," meaning that they do not deviate from their prescribed protocols.

We recall that when the row and column players play x and y respectively, the row player observes $-Ay$ and the column player observes $x^T A$. This enables the row and column players to solve minimization problems involving Ay and $x^T A$, respectively. The

HONESTEGTDYNAMICS protocol is a direct decoupling of the EGT algorithm.

We illustrate this decoupling idea by an example. The EGT algorithm requires solving the following optimization problem:

$$\check{x} := \arg \max_{x \in \Delta_n} (-x^T A y^k - \mu_n^k d_n(x)),$$

where $d_n(\cdot)$ is a function, μ_n^k is a constant known by the row player, and y^k is a strategy known by the column player. We can implement this maximization distributedly by instructing the row player to play x^k (a strategy computed earlier) and the column player to play y^k . The row player observes the loss vector $-Ay^k$, and he can then use local computation to compute $\check{x}$.

The HONESTEGTDYNAMICS protocol decouples the EGT algorithm exploiting this idea. We present the entire protocol in Appendix B. In this appendix, we also prove that the average payoffs of this protocol converge to the Nash equilibrium value with rate $O(\frac{\log T}{T})$.⁹

4 No-Regret Game Dynamics

We use the HONESTEGTDYNAMICS protocol as a starting block to design a no-regret protocol.

4.1 The No-Regret Property in Game Dynamics. We restate the *no-regret property* from Section 1.1 in the context of repeated zero-sum player interactions and define the *honest no-regret* property, a restriction of the no-regret property to the case where neither player is allowed to deviate from a prescribed protocol.

DEFINITION 4.1. *Fix a zero-sum game $(-A, A)_{n \times m}$ and a distributed protocol, specifying directions for the strategy that each player should chose at every time step given his observed payoff vectors. We call the protocol honest no-regret if it satisfies the following property: For all $\delta > 0$, there exists a T such that for all $T' > T$ and infinite sequences of strategies $(x_1, x_2, \dots)$ and $(y_1, y_2, \dots)$ resulting when the row and column players both follow the protocol:*

$$(4.1) \quad \frac{1}{T'} \sum_{t=1}^{T'} (-x_t^T A y_t) \geq \max_{i \in [n]} \frac{1}{T'} \sum_{t=1}^{T'} -(e_i)^T A y_t - \delta$$

$$(4.2) \quad \frac{1}{T'} \sum_{t=1}^{T'} (x_t^T A y_t) \geq \max_{i \in [m]} \frac{1}{T'} \sum_{t=1}^{T'} x_t^T A e_i - \delta.$$

We call the protocol no-regret for the column player if it satisfies the following property: For all $\delta > 0$,

⁹The proof of this convergence is not necessary for the remainder of the paper, since our later protocols will be simpler to analyze directly. We give it for completeness.

there exists a T such that for all $T' > T$ and infinite sequences of moves $(x_1, x_2, \dots)$ and $(y_1, y_2, \dots)$ resulting when the column player follows the protocol and the row player behaves arbitrarily, (4.2) is satisfied. We define similarly what it means for a protocol to be no-regret for the row player. We say that a protocol is no-regret if it is no-regret for both players.

The no-regret properties state that by following the protocol, a player's payoffs will not be significantly worse than the payoff that any single deterministic strategy would have achieved against the opponent's sequence of strategies.

We already argued that the average payoffs in the HONESTEGTDYNAMICS converge to the value of the game. However, this is not tantamount to the protocol being honest no-regret.¹⁰ To exemplify what goes wrong in our setting, in lines 17-18 of the protocol, the column player plays the strategy obtained by solving the following program, given the observed payoff vector $\hat{x}^T A$ induced by the strategy $\hat{x}$ of the other player.

$$\hat{y} := \arg \max_y (\hat{x}^T A y - \mu_m^k d_m(y)).$$

It is possible that the vector $\hat{y}$ computed above differs significantly from the equilibrium strategy y^* of the column player, even if the row player has converged to his equilibrium strategy $\hat{x} = x^*$. For example, suppose that $\hat{x} = x^*$, and that y^* involves mixing between two pure strategies in a 99%-1% ratio. We know that any combination of the two pure strategies supported by y^* will be a "best response" to x^* . Therefore, the minimizer of the above expression may involve mixing in, for example, a 50%-50% ratio of these strategies (given the canonization term $-\mu_m^k d_m(y)$ in the objective function). Since $\hat{y}$ differs significantly from y^* , there might be some best response x' to $\hat{y}$ which performs significantly better than x^* performs against $\hat{y}$, and thus the protocol may end up not being honest no-regret for the row player. A similar argument shows that the protocol is not necessarily honest no-regret for the column player.

4.2 "Honest No-Regret" Protocols. We perform a simple modification to the HONESTEGTDYNAMICS protocol to make it honest no-regret. The idea is for the players to only ever play strategies which are very close to the strategies x^k and y^k maintained by the

EGT algorithm at round k , which—by Theorem 2.1—constitute an approximate Nash equilibrium with the approximation going to 0 with k . Thus, for example, instead of playing $\hat{y}$ in line 18 of HONESTEGTDYNAMICS, the column player will play $(1 - \delta_k)y^k + \delta_k \hat{y}$, where δ_k is a very small fraction (say, $\delta_k = \frac{1}{(k+1)^2}$). Since the row player has previously observed Ay^k , and since δ_k is known to both players, the row player can compute the value of $A\hat{y}$. Furthermore, we note that the payoff of the best response to $(1 - \delta_k)y^k + \delta_k \hat{y}$ is within $2|A|_{\max}\delta_k$ of the payoff of the best response to y^k . Hence, the extra regret introduced by the mixture goes down with the number of rounds k . Indeed, the honest no-regret property resulting from this modification follows from this observation and the fact that x^k and y^k converge to a Nash equilibrium in the EGT algorithm (Theorem 2.1). (We do not give an explicit description of the modified HONESTEGTDYNAMICS and the proof of its honest no-regret property, as we incorporate this modification to further modifications that follow.)

4.3 Presumed Bound on $\sqrt{\ln n \ln m}$. We now begin work towards designing a no-regret protocol. Recall from Theorem 2.1 that the convergence rate of the EGT algorithm, and thus the rate of decrease of the average regret of the protocol from Section 4.2, depends on the value of $\sqrt{\ln n \ln m}$. However, without knowing the dimensions of the game (i.e. without knowledge of $\sqrt{\ln n \ln m}$), the players are incapable of measuring if their regret is decreasing as it should be, were they playing against an honest opponent. And if they have no ability to detect dishonest behavior and counteract, they could potentially be tricked by an adversary and incur high regret. In an effort to make our dynamics robust to adversaries and obtain the desired no-regret property, we design in this section a protocol, BOUNDEDEGTDYNAMICS(b), which takes a presumed upper bound b on $\sqrt{\ln n \ln m}$ as an input. This protocol will be our building block towards obtaining a no-regret protocol in the next section.

The idea for BOUNDEDEGTDYNAMICS(b) is straightforward: since a presumed upper bound b on $\sqrt{\ln n \ln m}$ is decided, the players can compute an upper bound on how much their regret ought to be in each round of the Section 4.2 protocol, assuming that b was a correct bound. If a player's regret in a round is ever greater than this computed upper bound, the player can conclude that either $b < \sqrt{\ln n \ln m}$, or that the opponent has not honestly followed the protocol. In the BOUNDEDEGTDYNAMICS protocol, a participant can detect two different types of failures, "YIELD" and "QUIT," described below. Both of these failures are internal state updates to a player's private

¹⁰For an easy example of why these two are not equivalent, consider the rock-paper-scissors game. Let the row player continuously play the uniform strategy over rock, paper, and scissors, and let the column player continuously play rock. The average payoff of the players is 0, which is the value of the game, but the row player always has average regret bounded away from 0.

computations and are not communicated to the other player. The distinction between the types of detectable violations will be important in Section 4.4.

- **YIELD(s)**- A YIELD failure means that a violation of a convergence guarantee has been detected. (In an honest execution, this will be due to b being smaller than $\sqrt{\ln n \ln m}$.) Our protocol can be designed so that, whenever one player detects an YIELD failure, the other player detects the same YIELD failure. A YIELD failure has an associated value s , which is the smallest “presumed upper bound on $\sqrt{\ln n \ln m}$ ” which, had s been given as the input to BOUNDEDEGTDYNAMICS instead of b , the failure would not have been declared.¹¹
- **QUIT**- A QUIT failure occurs when the opponent has been caught cheating. For example, a QUIT failure occurs if the row player is supposed to play the same strategy twice in a row but the column player observes different loss vectors. Unlike a YIELD failure, which could be due to the presumed upper bound being incorrect, a QUIT failure is a definitive proof that the opponent has deviated from the protocol.

For the moment, we can imagine a player switching to the MWU algorithm if he ever detects a failure. Clearly, this is not the right thing to do as a failure is not always due to a dishonest opponent, so this will jeopardize the fast convergence in the case of honest players. To avoid this, we will specify the appropriate behavior more precisely in Section 4.4.

We explicitly state and analyze the BOUNDEDEGTDYNAMICS(b) protocol in detail in Appendix C. The main lemma that we show is the following regret bound:

LEMMA 4.1. *Let $(x_1, x_2, \dots)$ and $(y_1, y_2, \dots)$ be sequences of strategies played by the row and column players respectively, where the column player used the BOUNDEDEGTDYNAMICS(b) protocol to determine his moves at each step. (The row player may or may not have followed the protocol.) If, after the first T rounds, the column player has not yet detected a YIELD or QUIT failure, then*

$$\begin{aligned} \max_{i \in [m]} \frac{1}{T} \sum_{t=1}^T x_t^T A e_i &\leq \frac{1}{T} \sum_{t=1}^T x_t^T A y_t \\ &+ \frac{19|A|_{\max}}{T} + \frac{20|A|_{\max} b \ln(T+3)}{T}. \end{aligned}$$

The analogous result holds for the row player.

¹¹The returned value s will not be important in this section, but will be used in Section 4.4.

Note that the value of b does not affect the strategies played in an execution of the BOUNDEDEGTDYNAMICS(b) protocol where both players are honest, as long as $b > \sqrt{\ln n \ln m}$. In this case, no failures will ever be detected.

4.4 The NoRegretEGT Protocol. In this section, we design our final no-regret protocol, NOREGRETEGT. The idea is to use the BOUNDEDEGTDYNAMICS(b) protocol with successively larger values of b , which we will guess as upper bounds on $\sqrt{\ln n \ln m}$. Notice that if we ever have a QUIT failure in the BOUNDEDEGTDYNAMICS protocol, the failure is a definitive proof that one of the players is dishonest. In this case, we instruct the player detecting the failure to simply perform the MWU algorithm forever, obtaining low regret.

The main difficulty is how to deal with the YIELD failures. The naive approach of running the BOUNDEDEGTDYNAMICS algorithm and doubling the value of b at every YIELD failure is not sufficient; intuitively, because this approach is not taking extra care to account for the possibility that either the guess on b is too low, or that the opponent is dishonest in a way preventing the dynamics from converging. Our solution is this: every time we would increase the value of b , we first perform a number of rounds of the multiplicative weights update method for a carefully chosen period length. In particular, we ensure that b is never greater than $\sqrt[4]{T}$ (for reasons which become clear in the analysis).

Now we have the following: If both players are honest, then after finitely many YIELD failures, b becomes larger than $\sqrt{\ln n \ln m}$. From that point on, we observe a failure-free run of the BOUNDEDEGTDYNAMICS protocol. Since this execution is failure-free, we argue that after the original finite prefix of rounds the regret can be bounded by Lemma 4.1. The crucial observation is that, if one of the players is dishonest and repeatedly causes YIELD failures of the BOUNDEDEGTDYNAMICS protocol, then the number of rounds of the MWU algorithm will be overwhelmingly larger than the number of rounds of the BOUNDEDEGTDYNAMICS (given our careful choice of the MWU period lengths), and the no-regret guarantee will follow from the MWU algorithm’s no-regret guarantees.

We present the NOREGRETEGT protocol in detail in Appendix D. The key results are the following two theorems, proved in the appendix. Together they imply Theorem 1.1.

THEOREM 4.1. *If the column player follows the NOREGRETEGT protocol, his average regret over the first T rounds is at most $O\left(\frac{|A|_{\max} \sqrt{\ln m}}{\sqrt{T}}\right)$, regardless of the row player’s actions. Similarly, if the row player follows*

the NOREGRETEGT protocol, his average regret over the first T rounds is at most $O\left(\frac{|A|_{\max}\sqrt{\ln n}}{\sqrt{T}}\right)$, regardless of the column player's actions.

THEOREM 4.2. *If both players honestly follow the NOREGRETEGT protocol, then the column player's average regret over the first T rounds is at most*

$$O\left(\frac{|A|_{\max}\sqrt{\ln n \ln m \ln T}}{T} + \frac{|A|_{\max}(\ln m)^{3/2} \ln n}{T}\right)$$

and the row player's average regret over the first T rounds is at most

$$O\left(\frac{|A|_{\max}\sqrt{\ln n \ln m \ln T}}{T} + \frac{|A|_{\max}(\ln n)^{3/2} \ln m}{T}\right).$$

5 Lower Bounds on Optimal Convergence Rate

In this section, we prove Theorem 1.2. The main idea is that since the players do not know the payoff matrix A of the zero-sum game, it is unlikely that their historical average strategies will converge to a Nash equilibrium very fast. In particular, the players are unlikely to play a Nash equilibrium in the first round and the error from that round can only be eliminated at a rate of $\Omega(1/T)$, forcing the $\Omega(1/T)$ convergence rate for the average payoffs and average strategies to the min-max solution.

Proof. [Proof of Theorem 1.2] We show that there exists a set of zero-sum games such that when a zero-sum game is selected randomly from the set, any strongly-uncoupled distributed protocol's convergence to the corresponding value of the game is $\Omega(1/T)$ with high probability. We assume that n and m are at least 2 to avoid degenerate cases. For $i = 1, \dots, n$, let A_i be the all-ones matrix except the i -th row which is the all zero vector. Note that the Nash equilibrium value of the game $(-A_i, A_i)$ is 0 for both players, and that all Nash equilibria are of the form (e_i, y) , where e_i is the deterministic strategy of choosing the i -th expert and $y \in \Delta_m$. Given any strongly-uncoupled protocol, consider choosing a game uniformly at random from the set $\mathcal{A} = \{(-A_1, A_1), \dots, (-A_n, A_n)\}$. Since the row player does not know the payoff matrix A in advance, the strategies x_1 and y_1 played in the first round of the protocol will have expected payoff $E[(x_1)^T(-A)y_1] = -1 + 1/n$. (Thus, the first-round payoff is at most $-1/3$ with probability at least $1 - \frac{3}{2n} \geq 1/4$.) Since the Nash equilibrium value of the game is 0, and the row player's payoffs are never strictly positive, the average payoffs $\frac{1}{T} \sum_t x_t^T(-A)y_t$ and $\frac{1}{T} \sum_t x_t^T A y_t$ converge to 0 (the value of the game) at expected rate no faster than $\Omega(1/T)$ in the number T of rounds. A similar argument

can be applied to bound on the rate that average strategies can converge to a min-max equilibrium in strongly-uncoupled dynamics.

Acknowledgements. We thank Robert Kleinberg for useful discussions.

References

- [1] I. Adler. On the Equivalence of Linear Programming Problems and Zero-Sum Games. *Optimization Online*, 2010.
- [2] F. Brandt, F. Fischer, and P. Harrenstein. On the Rate of Convergence of Fictitious Play. In *3rd International Symposium on Algorithmic Game Theory*, 2010.
- [3] G. W. Brown. Iterative solution of games by fictitious play. *Activity analysis of production and allocation*, 1951.
- [4] N. Cesa-Bianchi and G. Lugosi. *Prediction, learning, and games*. Cambridge University Press, 2006.
- [5] Y. Freund and R. Schapire. Adaptive Game Playing Using Multiplicative Weights. *Games and Economic Behavior*, 29:79–103, 1999.
- [6] A. Gilpin, J. Peña, and T. Sandholm. First-Order Algorithm With $O(\ln(1/\epsilon))$ Convergence for ϵ -Equilibrium in Two-Person Zero-Sum games. In *Proceedings of the 23rd National Conference on Artificial Intelligence*, 2008.
- [7] A. Gilpin, S. Hoda, J. Peña, and T. Sandholm. Gradient-based Algorithms for finding Nash Equilibria in Extensive Form Games. In *Proceedings of the Eighteenth International Conference on Game Theory*, 2007.
- [8] A. Gilpin, S. Hoda, J. Peña, and T. Sandholm. Smoothing Techniques for Computing Nash Equilibria of Sequential Games. *Optimization Online*, 2008.
- [9] S. Hart and A. Mas-Colell. Uncoupled Dynamics Do Not Lead to Nash Equilibrium. *American Economic Review*, 93:1830–1836, 2003.
- [10] N. Karmarkar. A New Polynomial-Time Algorithm for Linear Programming. In *Proceedings of the 16th Annual ACM Symposium on Theory of Computing*, 1984.
- [11] L. G. Khachiyan. A Polynomial Algorithm in Linear Programming. *Soviet Math. Dokl.*, 20(1):191–194, 1979.
- [12] G. Lan, Z. Lu, and R. Monteiro. Primal-Dual First-Order Methods with $O(1/\epsilon)$ iteration-complexity for cone programming. *Math. Program., Ser. A*, 2009.
- [13] N. Littlestone and M. Warmuth. The Weighted Majority Algorithm. *Information and Computation*, 108:212–261, 1994.
- [14] R. B. Myerson. Nash Equilibrium and the History of Economic Theory. *Journal of Economic Literature*, 1999.
- [15] J. Nash. Noncooperative Games. *Ann. Math.*, 54:289–295, 1951.

- [16] Y. Nesterov. Excessive Gap Technique in Nonsmooth Convex Minimization. *SIAM J. on Optimization* 16(1):235–249, May 2005.
- [17] Y. Nesterov. Smooth Minimization of Non-Smooth Functions. *Math. Program.*, 103(1):127–152, May 2005.
- [18] J. von Neumann. Zur Theorie der Gesellschaftsspiele. *Math. Annalen*, 100:295–320, 1928.
- [19] J. von Neumann and O. Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, 1944.
- [20] J. Robinson. An iterative method of solving a game. *Annals of mathematics*, 1951.

A Nesterov’s EGT Algorithm

In this appendix, we explain the ideas behind the Excessive Gap Technique (EGT) algorithm and we show how this algorithm can be used to compute approximate Nash equilibria in two-player zero-sum games. Before we discuss the algorithm itself, we introduce some necessary background terminology.

A.1 Choice of Norm. When we perform Nesterov’s algorithm, we will use norms $\|\cdot\|_n$ and $\|\cdot\|_m$ on the spaces Δ_n and Δ_m , respectively.¹² With respect to the norms $\|\cdot\|_n$ and $\|\cdot\|_m$ chosen above, we define the *norm* of A to be

$$\|A\|_{n,m} = \max_{x,y} \{x^T A y : \|x\|_n = 1, \|y\|_m = 1\}.$$

In this paper, we will choose to use ℓ_1 norms on Δ_n and Δ_m , in which case $\|A\|_{n,m} = |A|_{max}$, the largest absolute value of an entry of A .

A.2 Choice of Prox Function. In addition to choosing norms on Δ_n and Δ_m , we also choose smooth *prox-functions*, $d_n : \Delta_n \rightarrow \mathbb{R}$ and $d_m : \Delta_m \rightarrow \mathbb{R}$ which are strongly convex with convexity parameters $\sigma_n > 0$ and $\sigma_m > 0$, respectively.¹³ These prox functions will be used to construct the smooth approximations of f and ϕ . Notice that the strong convexity of our prox functions depends on our choice of norms $\|\cdot\|_n$ and $\|\cdot\|_m$. Without loss of generality, we will assume that d_n and d_m have minimum value 0.

Furthermore, we assume that the prox functions d_n and d_m are bounded on the simplex. Thus, there exist D_n and D_m such that

$$\max_{x \in \Delta_n} d_n(x) \leq D_n$$

¹²We use the notation Δ_n to represent the n -dimensional simplex.

¹³Recall that d_m is strongly convex with parameter σ_m if, for all v and $w \in \Delta_m$,

$$(\nabla d_m(v) - \nabla d_m(w))^T (v - w) \geq \sigma_m \|v - w\|_m^2.$$

and

$$\max_{y \in \Delta_m} d_m(y) \leq D_m.$$

A.3 Approximating f and ϕ by Smooth Functions. We will approximate f and ϕ by smooth functions f_{μ_m} and ϕ_{μ_n} , where μ_m and μ_n are smoothing parameters. (These parameters will change during the execution of the algorithm.) Given our choice of norms and prox functions above, we define

$$f_{\mu_m}(x) = \max_{v \in \Delta_m} x^T A v - \mu_m d_m(v)$$

$$\phi_{\mu_n}(y) = \min_{u \in \Delta_n} u^T A y + \mu_n d_n(u).$$

We see that for small values of μ , the functions will be a very close approximation to their non-smooth counterparts. We observe that since d_n and d_m are strongly convex functions, the optimizers of the above expressions are unique.

As discussed above, for all $x \in \Delta_n$ and $y \in \Delta_m$ it is the case that $\phi(y) \leq f(x)$. Since $f_{\mu_m}(x) \leq f(x)$ and $\phi_{\mu_n}(y) \geq \phi(y)$ for all x and y , it is possible that some choice of values μ_n , μ_m , x and y may satisfy the *excessive gap condition* of $f_{\mu_m}(x) \leq \phi_{\mu_n}(y)$. The key point behind the excessive gap condition is the following simple lemma from [16]:

LEMMA A.1. *Suppose that*

$$f_{\mu_m}(x) \leq \phi_{\mu_n}(y).$$

Then

$$f(x) - \phi(y) \leq \mu_n D_n + \mu_m D_m.$$

Proof. For any $x \in \Delta_n$ and $y \in \Delta_m$, we have $f_{\mu_m}(x) \geq f(x) - \mu_m D_m$ and $\phi_{\mu_n}(y) \leq \phi(y) + \mu_n D_n$. Therefore

$$f(x) - \phi(y) \leq f_{\mu_m}(x) + \mu_m D_m - \phi_{\mu_n}(y) + \mu_n D_n$$

and the lemma follows immediately.

In the algorithms which follow, we will attempt to find x and y such that $f_{\mu_m}(x) \leq \phi_{\mu_n}(y)$ for μ_n , μ_m small.

A.4 Excessive Gap Technique (EGT) Algorithm. We now present the gradient-based excessive gap technique from [16] in the context of zero-sum games (see [7], [8]). The main idea behind the excessive gap technique is to gradually lower μ_m and μ_n while updating values of x and y such that the invariant $f_{\mu_m}(x) \leq \phi_{\mu_n}(y)$ holds.

The following gradient-based algorithm uses the techniques of [16], and is presented here in the form

from [8]. In Appendix B, we show how to implement this algorithm by game dynamics.

In the algorithm which follows, we frequently encounter terms of the form

$$d_m(x) - x^T \nabla d_m(\hat{x}).$$

We intuitively interpret these terms by noting that

$$\xi_m(\hat{x}, x) = d_m(x) - d_m(\hat{x}) - (x - \hat{x})^T \nabla d_m(\hat{x})$$

is the *Bregman distance* between $\hat{x}$ and x . Thus, when $\hat{x}$ is fixed, looking at an expression such as

$$\arg \max_{x \in \Delta_n} -x^T A y^0 + \mu_n^0 (x^T \nabla d_n(\hat{x}) - d_n(x))$$

should be interpreted as looking for x with small Bregman distance from $\hat{x}$ which makes $-x^T A y^0$ large. Loosely speaking, we may colloquially refer to the optimal x above as a “smoothed best response” to $A y^0$.

The key point to this algorithm is the following theorem, from [16]

THEOREM A.1. *The x^k and y^k generated by the EGT algorithm satisfy*

$$f(x^k) - \phi(y^k) \leq \frac{4\|A\|_{n,m}}{k+1} \sqrt{\frac{D_n D_m}{\sigma_n \sigma_m}}.$$

A.5 Entropy Prox Function and the ℓ_1 Norm.

When we simulate the EGT algorithm with game dynamics, we will choose to use the ℓ_1 norm and the entropy prox function, as defined below. (This choice of norm and prox function was mentioned in [17].)

$$d_n(x) = \ln n + \sum_{i=1}^n x_i \ln x_i$$

$$d_m(y) = \ln m + \sum_{j=1}^m y_j \ln y_j$$

$$\|x\|_n = \sum_{i=1}^n |x_i|$$

$$\|y\|_m = \sum_{j=1}^m |y_j|$$

From Lemma 4.3 of [17], we know that the above choice of norms and prox functions satisfy:

$$\sigma_n = \sigma_m = 1$$

$$D_n = \ln n$$

$$D_m = \ln m$$

$$\|A\|_{n,m} = |A|,$$

```

1: function EGT
2:    $\mu_n^0 := \mu_m^0 := \frac{\|A\|_{n,m}}{\sqrt{\sigma_n \sigma_m}}$ 
3:    $\hat{x} := \arg \min_{x \in \Delta_n} d_n(x)$ 
4:    $y^0 := \arg \max_{y \in \Delta_m} \hat{x}^T A y - \mu_m^0 d_m(y)$ 
5:    $x^0 := \arg \max_{x \in \Delta_n} -x^T A y^0 + \mu_n^0 (x^T \nabla d_n(\hat{x}) - d_n(x))$ 
6:
7:   for  $k = 0, 1, 2, \dots$  do
8:      $\tau := \frac{2}{k+3}$ 
9:
10:    if  $k$  is even then /* Shrink  $\mu_n$  */
11:       $\check{x} := \arg \max_{x \in \Delta_n} -x^T A y^k - \mu_n^k d_n(x)$ 
12:       $\hat{x} := (1 - \tau)x^k + \tau \check{x}$ 
13:       $\hat{y} := \arg \max_{y \in \Delta_m} \hat{x}^T A y - \mu_m^k d_m(y)$ 
14:       $\tilde{x} := \arg \max_{x \in \Delta_n} -x^T A \hat{y} + \mu_n^k x^T \nabla d_n(\check{x}) - \mu_n^k d_n(x)$ 
15:       $y^{k+1} := (1 - \tau)y^k + \tau \hat{y}$ 
16:       $x^{k+1} := (1 - \tau)x^k + \tau \tilde{x}$ 
17:       $\mu_n^{k+1} := (1 - \tau)\mu_n^k$ 
18:       $\mu_m^{k+1} := \mu_m^k$ 
19:    end if
20:
21:    if  $k$  is odd then /* Shrink  $\mu_m$  */
22:       $\check{y} := \arg \max_{y \in \Delta_m} y^T A^T x^k - \mu_m^k d_m(y)$ 
23:       $\hat{y} := (1 - \tau)y^k + \tau \check{y}$ 
24:       $\hat{x} := \arg \max_{x \in \Delta_n} -x^T A \hat{y} - \mu_n^k d_n(x)$ 
25:       $\tilde{y} := \arg \max_{y \in \Delta_m} \frac{\tau}{1-\tau} y^T A^T \hat{x} + \mu_m^k y^T \nabla d_m(\check{y}) - \mu_m^k d_m(y)$ 
26:       $x^{k+1} := (1 - \tau)x^k + \tau \hat{x}$ 
27:       $y^{k+1} := (1 - \tau)y^k + \tau \tilde{y}$ 
28:       $\mu_m^{k+1} := (1 - \tau)\mu_m^k$ 
29:       $\mu_n^{k+1} := \mu_n^k$ 
30:    end if
31:  end for
32: end function

```

where $|A|$ is the largest absolute value entry of A . (In the EGT algorithm, it suffices to replace $\|A\|_{n,m}$ with $|A|_{max}$, an upper bound of $|A|$. When we make this change, we will simply replace $\|A\|_{n,m}$ with $|A|_{max}$ in the above theorem.)

There are three main benefits of choosing these prox functions. The first reason is that this choice will make our convergence bounds depend on the same parameters as the MWU convergence bounds, and thus it will be easy to compare the convergence rates of these technique.

The second reason is that in the first step of the EGT algorithm, we set $\mu_n^0 := \mu_m^0 := \frac{\|A\|_{n,m}}{\sqrt{\sigma_n \sigma_m}}$. Since $\sigma_n = \sigma_m = 1$ under our choice of prox functions and ℓ_1 norm, this step of the algorithm simply becomes

$$\mu_n^0 := \mu_m^0 := |A|_{max},$$

which is a known constant.

The third reason is that all of the required optimizations have simple closed-form solutions. In particular, our algorithm requires us to solve optimization problems of the form

$$\arg \max_{x \in \Delta_n} x^T s - \mu_n d_n(x)$$

where $s \in \mathbb{R}^n$ is some fixed vector. In this case, the solution has a closed form (see [17]). The solution is the vector x , with j^{th} component

$$x_j = \frac{e^{s_j/\mu_n}}{\sum_{i=1}^n e^{s_i/\mu_n}}.$$

The analogous result holds for optimizations over $y \in \Delta_m$.

B The Honest EGT Dynamics Protocol

In this appendix, we present the entirety of the HONESTEGTDYNAMICS protocol, introduced in Section 3, and compute convergence bounds for the average payoffs. Note that throughout the appendix, we present the HONESTEGTDYNAMICS protocol, and protocols which follow, as a single block of pseudocode containing instructions for both row and column players. However, this presentation is purely for notational convenience, and our pseudocode can clearly be written as a protocol for the row player and a separate protocol for the column player.

For notational purposes, most lines of our pseudocode begin with either a “ R ” or a “ C ” marker. These symbols refer to instructions performed by the row or column player, respectively. A line which begins with the “ R, C ” marker is a computation performed independently by both players. An instruction such as “PLAY

$x^T Ay$ ” is shorthand for an instruction of “PLAY x ” in the row player’s protocol, and “PLAY y ” in the column player’s protocol.

We compute convergence bounds for the average payoff in the HONESTEGTDYNAMICS protocol, assuming that both players honestly follow the protocol. These bounds are slightly more difficult to compute than the bounds for the BOUNDEDEGTDYNAMICS(∞) protocol (which also converges quickly towards a Nash equilibrium when both players follow the protocol.) We include these bounds on the (less efficient) HONESTEGTDYNAMICS protocol for the sake of completeness.

We will use Theorem 2.1 to bound the payoffs every time the players play a round of the game. Our goal is to prove that the average payoffs in HONESTEGTDYNAMICS converge to the Nash Equilibrium value quickly (with convergence rate $O(\frac{\ln T}{T})$).

In what follows, we let $P = (x^*)^T Ay^*$ be the Nash equilibrium payoff (for the row player) of the game. For ease of notation, in the analysis which follows we let

$$\epsilon_k = \frac{4|A|_{max}\sqrt{\ln n \ln m}}{k+1}.$$

We now have the following bounds on the payoffs, where we analyze each line of HONESTEGTDYNAMICS separately:

- Line 5- We simply bound this payoff by

$$-|A|_{max} \leq \hat{x}^T A \bar{y} \leq |A|_{max}.$$

- Line 11

Using Theorem 2.1, we have

$$P - \epsilon_k \leq (x^k)^T Ay^k \leq P + \epsilon_k.$$

- Line 16-

We notice that $\hat{x}^T Ay^k \leq (1 - \tau)(x^k)^T Ay^k + \tau|A|_{max}$. This will enable us to bound on $\hat{x}^T Ay^k$ by using Theorem 2.1. Note that

$$\begin{aligned} \hat{x}^T Ay^k &\leq (1 - \tau)(P + \epsilon_k) + \tau|A|_{max} \\ &\leq P + \tau|A|_{max} + (1 - \tau)\epsilon_k + \tau|A|_{max} \\ &\leq P + \epsilon_k + \frac{4|A|_{max}}{k+3}. \end{aligned}$$

Therefore, we have the bounds

$$P - \epsilon_k \leq \hat{x}^T Ay^k \leq P + \epsilon_k + \frac{4|A|_{max}}{k+3}.$$

```

1: function HONEST EGT DYNAMICS
2:    $R : \mu_n^0 := |A|_{max}$     $C : \mu_m^0 := |A|_{max}$ 
3:    $R : \hat{x} := \arg \min_{x \in \Delta_n} d_n(x)$ 
4:    $C : \text{Pick } \bar{y} \in \Delta_m \text{ arbitrary}$ 
5:    $\text{PLAY: } \hat{x}^T A \bar{y}$ 
6:    $C : y^0 := \arg \max_{y \in \Delta_m} \hat{x}^T A y - \mu_m^0 d_m(y)$ 
7:    $R :$ 

    $x^0 := \arg \max_{x \in \Delta_n} -x^T A y^0 + \mu_n^0 (x^T \nabla d_n(\hat{x}) - d_n(x))$ 

8:
9:   for  $k = 0, 1, 2, \dots$  do
10:     $R, C : \tau := \frac{2}{k+3}$ 
11:     $\text{PLAY: } (x^k)^T A y^k$ 
12:
13:    if  $k$  is even then /* Shrink  $\mu_n$  */
14:       $R : \check{x} := \arg \max_{x \in \Delta_n} -x^T A y^k - \mu_n^k d_n(x)$ 
15:       $R : \hat{x} := (1 - \tau)x^k + \tau \check{x}$ 
16:       $\text{PLAY: } \hat{x}^T A y^k$ 
17:       $C : \hat{y} := \arg \max_{y \in \Delta_m} \hat{x}^T A y - \mu_m^k d_m(y)$ 
18:       $\text{PLAY } \hat{x}^T A \hat{y}$ 
19:       $R :$ 

       $x^{k+1} := (1 - \tau)x^k + \tau(\arg \max_{x \in \Delta_n} \{-\frac{\tau}{1 - \tau} x^T A \hat{y}$ 
         $+ \mu_n^k (x^T \nabla d_n(\check{x}) - d_n(x))\})$ 

20:       $C : y^{k+1} := (1 - \tau)y^k + \tau \hat{y}$ 
21:       $R : \mu_n^{k+1} := (1 - \tau)\mu_n^k$ 
22:       $C : \mu_m^{k+1} := \mu_m^k$ 
23:    end if

24:
25:    if  $k$  is odd then /* Shrink  $\mu_m$  */
26:       $C : \check{y} := \arg \max_{y \in \Delta_m} y^T A^T x^k - \mu_m^k d_m(y)$ 
27:       $C : \hat{y} := (1 - \tau)y^k + \tau \check{y}$ 
28:       $\text{PLAY: } (x^k)^T A \hat{y}$ 
29:       $R : \hat{x} := \arg \max_{x \in \Delta_n} -x^T A \hat{y} - \mu_n^k d_n(x)$ 
30:       $\text{PLAY: } \hat{x}^T A \hat{y}$ 
31:       $C :$ 

       $y^{k+1} := (1 - \tau)y^k + \tau(\arg \max_{y \in \Delta_m} \{\frac{\tau}{1 - \tau} y^T A^T \hat{x}$ 
         $+ \mu_m^k (y^T \nabla d_m(\check{y}) - d_m(y))\})$ 

32:       $R : x^{k+1} := (1 - \tau)x^k + \tau \hat{x}$ 
33:       $C : \mu_m^{k+1} := (1 - \tau)\mu_m^k$ 
34:       $R : \mu_n^{k+1} := \mu_n^k$ 
35:    end if
36:  end for
37: end function

```

- Line 18- We notice that, since $\hat{y} := \arg \max_{y \in \Delta_m} \hat{x}^T A y - \mu_m^k d_m(y)$, we have

$$\hat{x}^T A \hat{y} \geq \arg \max_{y \in \Delta_m} \{\hat{x}^T A y\} - \mu_m^k d_m$$

Furthermore, since $\arg \max_{y \in \Delta_m} \{\hat{x}^T A y\} \geq P$, this gives us the bound

$$P - \mu_m^k \ln m \leq \hat{x}^T A \hat{y}.$$

Now we determine the value of μ_m^k . Notice that k is even at Line 18, and therefore

$$\begin{aligned} \mu_m^k &= \mu_m^0 \cdot \prod_{i=1, i \text{ odd}}^{k-1} (1 - \frac{2}{i+3}) \\ &= \mu_m^0 \cdot \prod_{i=1, i \text{ odd}}^{k-1} \frac{i+1}{i+3} \\ &= \frac{2|A|_{max}}{k+2}. \end{aligned}$$

To obtain an upper bound, we notice that

$$\hat{x} := (1 - \tau)x^k + \tau(\arg \max_{x \in \Delta_n} \{-x^T A y^k - \mu_n^k d_n(x)\}).$$

Therefore,

$$\begin{aligned} \hat{x}^T A \hat{y} &\leq (1 - \tau)(x^k)^T A \hat{y} + \tau |A|_{max} \\ &\leq P + \epsilon_k + 2\tau |A|_{max} \\ &= P + \epsilon_k + \frac{4|A|_{max}}{k+3}. \end{aligned}$$

Putting these bounds together, we have

$$P - \frac{2|A|_{max} \ln m}{k+2} \leq \hat{x}^T A \hat{y} \leq P + \epsilon_k + \frac{4|A|_{max}}{k+3}.$$

- Line 28-

By the same analysis as Line 16, we have

$$P - \epsilon_k - \frac{4|A|_{max}}{k+3} \leq (x^k)^T A \hat{y} \leq P + \epsilon_k.$$

- Line 30-

The analysis is nearly identical to the analysis from Line 18. The only difference is that, since k is odd, we have

$$\begin{aligned} \mu_n^k &= \mu_n^0 \cdot \prod_{i=0, i \text{ even}}^{k-1} \frac{i+1}{i+3} \\ &= \frac{|A|_{max}}{k+2}. \end{aligned}$$

Therefore, we have the bound

$$P - \epsilon_k - \frac{4|A|_{max}}{k+3} \leq \hat{x}^T A \hat{y} \leq P + \frac{|A|_{max} \ln n}{k+2}.$$

By using these bounds, we can obtain the following lemma, which we prove below:

LEMMA B.1. *For all $K \geq 1$, the average payoff of playing the HONESTEGTDYNAMICS for a total of $3K+1$ rounds is bounded by*

$$\begin{aligned} P - \frac{|A|_m}{K} - \frac{24|A|_m \ln(K+1)}{3K+1} (\sqrt{\ln n \ln m} + \ln m + 1) \\ \leq \text{Average Payoff} \leq \\ P + \frac{|A|_m}{K} + \frac{24|A|_m \ln(K+1)}{3K+1} (\sqrt{\ln n \ln m} + \ln n + 1) \end{aligned}$$

where $P = (x^*)^T A y^*$ is the Nash equilibrium value of the game and $|A|_m = |A|_{max}$.

Comparing this lemma to Corollary 1.1, we observe that the average payoffs of HONESTEGTDYNAMICS have better asymptotic convergence (in the number of rounds played) to a Nash equilibrium than the MWU algorithm.

Proof. We see that we can lower bound the sum of the three payoffs obtained for any fixed value of k (the payoffs received in lines 11, 16, and 18 if k is even, and in lines 11, 28, and 30 if k is odd) by

$$3P - 3\epsilon_k - \frac{8|A|_{max}}{k+3} - \frac{2|A|_{max} \ln m}{k+2}.$$

Therefore, we lower bound the average payoff by

$$\begin{aligned} & \frac{1}{3K+1} \left(-|A|_{max} + \sum_{k=0}^{K-1} (3P - 3\epsilon_k - \frac{8|A|_{max}}{k+3} - \frac{2|A|_{max} \ln m}{k+2}) \right) \\ & \geq \frac{1}{3K+1} \left(-|A|_{max} + 3KP - 3 \sum_{k=0}^{K-1} \epsilon_k - |A|_{max} (8 + 2 \ln m) \sum_{k=0}^{K-1} \frac{1}{k+2} \right) \\ & \geq \frac{1}{3K+1} \left(-|A|_{max} + 3KP - |A|_{max} (12\sqrt{\ln n \ln m} + 8 + 2 \ln m) \sum_{k=0}^{K-1} \frac{1}{k+1} \right) \\ & \geq \frac{1}{3K+1} \left((-2|A|_{max} + (3K+1)P) - |A|_{max} (1 + \ln K) (12\sqrt{\ln n \ln m} + 8 + 2 \ln m) \right) \\ & \geq \frac{1}{3K+1} \left(-2|A|_{max} + (3K+1)P - |A|_{max} (2 \ln(K+1)) (12\sqrt{\ln n \ln m} + 8 + 2 \ln m) \right) \\ & = P - \frac{1}{3K+1} \left\{ 2|A|_{max} + \left(24|A|_{max} \sqrt{\ln n \ln m} + 4|A|_{max} \ln m + 16|A|_{max} \right) \ln(K+1) \right\}. \end{aligned}$$

Similarly, we can upper bound the sum of the three payoffs obtained for any fixed value of k by

$$3P + 3\epsilon_k + \frac{8|A|_{max}}{k+3} + \frac{|A|_{max} \ln n}{k+2}.$$

Therefore, by similar calculations as to those above, we can upper bound the average payoff received over the first $3K+1$ rounds by

$$\begin{aligned} P + \frac{1}{3K+1} \left\{ 2|A|_{max} + \left(24|A|_{max} \sqrt{\ln n \ln m} + 2|A|_{max} \ln n + 16|A|_{max} \right) \ln(K+1) \right\}. \end{aligned}$$

The statement of the lemma follows.

C The BoundedEgtDynamics(b) Protocol

In this appendix, we describe and analyze the BOUNDEDegTDYNAMICS protocol in detail. For clarity, we break the algorithm apart into subroutines. The overall structure is very similar to the HONESTEGTDYNAMICS protocol, but the players continually check for evidence that the opponent might have deviated from his instructions.

C.1 The Initialization Routine. We first describe the INITIALIZATION routine. This routine sets the values of x^0 , y^0 , μ_n^0 , and μ_m^0 . It is identical to lines 2 through 7 of the HONESTEGTDYNAMICS protocol.

```

1: function INITIALIZATION /*  $R$  sets  $x^0$  and  $\mu_n^0$ .  $C$ 
   sets  $y^0$  and  $\mu_m^0$  */
2:    $R : \mu_n^0 := |A|_{max}$     $C : \mu_m^0 := |A|_{max}$ 
3:    $R : \hat{x} := \arg \min_{x \in \Delta_n} d_n(x)$ 
4:    $C : \text{Pick } \bar{y} \in \Delta_m \text{ arbitrary}$ 
5:    $\text{PLAY} : \hat{x}^T A \bar{y}$ 
6:    $C : y^0 := \arg \max_{y \in \Delta_m} \hat{x}^T A y - \mu_m^0 d_m(y)$ 
7:    $R : x^0 := \arg \max_{x \in \Delta_n} -x^T A y^0 + \mu_n^0 (x^T \nabla d_n(\hat{x}) - d_n(x))$ 
8: end function

```

C.2 The CheckConv Routine. We now describe the CHECKCONV routine. The goal of this routine is to verify if (x^k, y^k) is indeed an (additive) ϵ_k -approximate Nash equilibrium.¹⁴ In this routine, the row player is given an opportunity to play a move which gives him more than ϵ_k payoff against y^k than he would obtain by playing x^k . If he cannot find such a move, then the column player is given a chance. If (x^k, y^k) is indeed an ϵ_k -approximate Nash equilibrium, then this routine will simply consist of three rounds of $(x^k)^T A y^k$.

C.3 The SafePlay Routines. The key to making our protocol no-regret is the SAFEPLAY routines. These routines are used to replace instructions such as “PLAY $\hat{x}^T A y^k$ ” from the HONESTEGTDYNAMICS protocol. In these routines, the players verify several properties, including that the payoffs fall within the expected bounds.

Before running the SAFEPLAY routines, we assume that $(x^k)^T A y^k$ has been already been played at some point in the protocol, so that the row player knows the loss vector $A y^k$ and the column player knows the loss vector $(x^k)^T A$. Both players know a value ϵ_k , and they currently believe (x^k, y^k) to be an ϵ_k -approximate Nash equilibrium.¹⁵ They both know $\hat{P}$, the value of $(x^k)^T A y^k$, which they will use as an estimate of the Nash equilibrium value.

The idea of the routines is that instead of playing $\hat{x}$, the row player will play $\delta_k \hat{x} + (1 - \delta_k) x^k$, where δ_k is some (small) value known to both players. Since the

column player will have already observed the loss vector $(x^k)^T A$, we will be able to determine the vector $\hat{x}^T A$.

We now define the SAFEPLAYROW routine (for the row player to convey a loss vector to the column player) and the SAFEPLAYCOL routine (for the column player to convey a loss vector to the row player).

```

1: function SAFEPLAYROW( $x, \hat{P}, \epsilon_k, \delta_k, (x^k)^T A, A y^k$ )
2:   /* Protocol for the row player to convey  $x^T A$  to
   the column player */
3:   /*  $(x^k)^T A$  is a loss vector previously observed
   by the column player */
4:   /*  $A y^k$  is a loss vector previously observed by
   the row player */
5:   /*  $\hat{P}, \epsilon_k, \delta_k$  known by both players */
6:    $\text{PLAY } (\delta_k x + (1 - \delta_k) x^k)^T A y^k$ . Call this value
    $p$ . Let  $u^T$  be the loss vector observed by the column
   player, and let  $v$  be the loss vector observed by the
   row player.
7:    $C : \text{Set } ans = \frac{u^T - (1 - \delta_k)(x^k)^T A}{\delta_k}$ 
8:    $C : \text{If any entry of } ans \text{ has absolute value greater}$ 
    $\text{than } |A|_{max}, \text{ QUIT.}$ 
9:    $R : \text{If } v \neq A y^k, \text{ QUIT.}$ 
10:   $R, C : \text{If } |\hat{P} - p| > \epsilon_k + 2|A|_{max} \delta_k, \text{ YIELD}(|\hat{P} -$ 
    $p| - 2|A|_{max} \delta_k).$ 
11:   $C : \text{Conclude that } x^T A = ans$ 
12: end function

```

Notice that the check on line 8 of the SAFEPLAYROW routine ensures that the loss vector u^T is very close to $(x^k)^T A$. This is a key property for showing that the protocol is no-regret (since it ensures that the payoff of a best response to the loss vector u^T and the payoff of a best response to the loss vector $(x^k)^T A$ differ by no more than $2\delta_k |A|_{max}$.) In particular, it means that y^k is very close to a best response to the loss vector u^T .

C.4 The BoundedEgtDynamics(b) Protocol. We now describe the BOUNDEDEGTDYNAMICS protocol using the above subroutines. This protocol is nearly identical in structure to the HONESTEGTDYNAMICS protocol. If a YIELD or QUIT failure is detected, the players switch to the MWU algorithm.

C.5 Bounding the Regret. We will show that the protocol is no-regret for the column player. (The analysis for the row player is nearly identical.) As a corollary, we will obtain a bound on the convergence rate of the average payoffs. We split our analysis into two cases. The first case is that the column player declares a YIELD or QUIT failure at some point in the algorithm. We notice that if a failure is detected, then

¹⁴More precisely, the row player verifies that his best response to y^k gives him no more than ϵ_k additional payoff over $-(x^k)^T A y^k$. The column player then checks an analogous property.

¹⁵These beliefs have been established by the CHECKCONV routine.

```

1: function CHECKCONV( $x^k, y^k, \epsilon_k$ ) /* Check if  $(x^k, y^k)$  is  $\epsilon_k$ -Nash.*/
2:   /* If no failures occur, returns the value of  $(x^k)^T A y^k$  */
3:   PLAY:  $(x^k)^T A y^k$ 
4:
5:   R:  $\dot{x} := \arg \max_{x \in \Delta_n} -x^T A y^k$ 
6:   R: If  $(-\dot{x}^T A y^k) > -(x^k)^T A y^k + \epsilon_k$ ,  $\ddot{x} := \dot{x}$ . Else  $\ddot{x} := x^k$ 
7:
8:   PLAY  $\ddot{x}^T A y^k$ 
9:
10:  R: If the observed loss vectors  $A y^k$  in lines 3 and 8 differ, QUIT.
11:  R, C: If  $\ddot{x}^T A y^k < ((x^k)^T A y^k) - \epsilon_k$ , YIELD( $\frac{((x^k)^T A y^k - \ddot{x}^T A y^k)(k+1)}{4|A|_{max}}$ ).
12:  C: If  $\ddot{x}^T A y^k \neq ((x^k)^T A y^k)$  and  $\ddot{x}^T A y^k \geq ((x^k)^T A y^k) - \epsilon_k$ , QUIT.
13:
14:  C:  $\dot{y} := \arg \max_{y \in \Delta_y} ((x^k)^T A y)$ 
15:  C: If  $(x^k)^T A \dot{y} > ((x^k)^T A y^k) + \epsilon_k$ ,  $\ddot{y} := \dot{y}$ . Else  $\ddot{y} := y^k$ 
16:
17:  PLAY  $(x^k)^T A \ddot{y}$ 
18:
19:  C: If the observed loss vectors  $(x^k)^T A$  in lines 3 and 17 differ, QUIT.
20:  R, C: If  $(x^k)^T A \ddot{y} > ((x^k)^T A y^k) + \epsilon_k$ , YIELD( $\frac{((x^k)^T A \ddot{y} - (x^k)^T A y^k)(k+1)}{4|A|_{max}}$ ).
21:  R: If  $(x^k)^T A \ddot{y} \neq ((x^k)^T A y^k)$  and  $(x^k)^T A \ddot{y} \leq ((x^k)^T A y^k) + \epsilon_k$ , QUIT.
22:  return  $(x^k)^T A y^k$ 
23: end function

```

```

1: function SAFEPLAYCOL( $y, \hat{P}, \epsilon_k, \delta_k, (x^k)^T A, A y^k$ )
2:   /* Protocol for the column player to convey  $A y$  to the row player */
3:   /*  $(x^k)^T A$  is a loss vector previously observed by the column player */
4:   /*  $A y^k$  is a loss vector previously observed by the row player */
5:   /*  $\hat{P}, \epsilon_k, \delta_k$  known by both players */
6:   PLAY  $(x^k)^T A(\delta_k y + (1 - \delta_k)y^k)$ . Call this value  $p$ . Let  $u^T$  be the loss vector observed by the column player, and let  $v$  be the loss vector observed by the row player.
7:   R: Set  $ans = \frac{v - (1 - \delta_k)A(y^k)}{\delta_k}$ 
8:   R: If any entry of  $ans$  has absolute value greater than  $|A|_{max}$ , QUIT.
9:   C: If  $u^T \neq (x^k)^T A$ , QUIT.
10:  R, C: If  $|\hat{P} - p| > \epsilon_k + 2|A|_{max}\delta_k$ , YIELD( $|\hat{P} - p| - 2|A|_{max}\delta_k$ ).
11:  R: Conclude that  $A y = ans$ 
12: end function

```

the column player switches to the MWU algorithm after some finite number of rounds. Therefore, the no-regret property will follow immediately from the fact that the MWU algorithm is a no-regret strategy.

Now we consider some execution of the protocol in which the column player never detects a failure and look at all sections of the BOUNDEDEGTDYNAMICS protocol where the game was played. In this analysis, we will prove a stronger claim than the no-regret property. Instead of showing that column player has no single strategy which would have performed significantly better against the opponent's historical average, we show that the column player has no sequences of strategies which would have performed significantly better against the opponent's strategy history. (Thus, if we were to tell the column player in advance all of opponent's moves in order, and allowed the column player to change his move from round to round, he would still not be able to perform significantly better.)

- Line 2 -The INITIALIZATION routine is only played once during the entire execution. We can lower bound the payoff received in this round by $-|A|_{max}$. By deviating, it is possible that the column player could changed his payoff to no more than $|A|_{max}$, and therefore the column player could have gained at most $2|A|_{max}$ by deviating.

```

1: function BOUNDEDEGTDYNAMICS( $b$ ) /*  $b$  is presumed upper bound on  $\sqrt{\ln n \ln m}$  */
2:   Run INITIALIZATION
3:
4:   while No YIELD or QUIT failures have occurred do
5:
6:     for  $k = 0, 1, 2, \dots$  do
7:        $R, C: \tau_k := \frac{2}{k+3}$ 
8:        $R, C: \epsilon_k := \frac{4|A|_{max}b}{k+1}$ 
9:        $R, C: \delta_k := \frac{1}{(k+1)^2}$ 
10:      Run CHECKCONV( $x^k, y^k, \epsilon_k$ ).  $R$  and  $C$  set  $\hat{P} := (x^k)^T A y^k$ 
11:
12:      if  $k$  is even then /* Shrink  $\mu_n$  */
13:         $R: \check{x} := \arg \max_{x \in \Delta_n} -x^T A y^k - \mu_n^k d_n(x)$ 
14:         $R: \hat{x} := (1 - \tau_k)x^k + \tau_k \check{x}$ 
15:        SAFEPLAYROW( $\hat{x}, \hat{P}, \epsilon_k, \delta_k, (x^k)^T A, A y^k$ )
16:         $C: \hat{y} := \arg \max_{y \in \Delta_m} \hat{x}^T A y - \mu_m^k d_m(y)$ 
17:        SAFEPLAYCOL( $\hat{y}, \hat{P}, \epsilon_k, \delta_k, (x^k)^T A, A y^k$ )
18:         $R: x^{k+1} := (1 - \tau_k)x^k + \tau_k(\arg \max_{x \in \Delta_n} \{-\frac{\tau_k}{1-\tau_k} x^T A \hat{y} + \mu_n^k(x^T \nabla d_n(\check{x}) - d_n(x))\})$ 
19:         $C: y^{k+1} := (1 - \tau_k)y^k + \tau_k \hat{y}$ 
20:         $R: \mu_n^{k+1} := (1 - \tau_k)\mu_n^k$ 
21:         $C: \mu_m^{k+1} := \mu_m^k$ 
22:      end if
23:
24:      if  $k$  is odd then /* Shrink  $\mu_m$  */
25:         $C: \check{y} := \arg \max_{y \in \Delta_m} y^T A^T x^k - \mu_m^k d_m(y)$ 
26:         $C: \hat{y} := (1 - \tau_k)y^k + \tau_k \check{y}$ 
27:        SAFEPLAYCOL( $\hat{y}, \hat{P}, \epsilon_k, \delta_k, (x^k)^T A, A y^k$ )
28:         $R: \hat{x} := \arg \max_{x \in \Delta_n} -x^T A \hat{y} - \mu_n^k d_n(x)$ 
29:        SAFEPLAYROW( $\hat{x}, \hat{P}, \epsilon_k, \delta_k, (x^k)^T A, A y^k$ )
30:         $C: y^{k+1} := (1 - \tau_k)y^k + \tau_k(\arg \max_{y \in \Delta_m} \{-\frac{\tau_k}{1-\tau_k} y^T A^T \hat{x} + \mu_m^k(y^T \nabla d_m(\check{y}) - d_m(y))\})$ 
31:         $R: x^{k+1} := (1 - \tau_k)x^k + \tau_k \hat{x}$ 
32:         $C: \mu_m^{k+1} := (1 - \tau_k)\mu_m^k$ 
33:         $R: \mu_n^{k+1} := \mu_n^k$ 
34:      end if
35:    end for
36:  end while
37:  Use the multiplicative weights update algorithm in all subsequent rounds.
38: end function

```

- Line 10 - Since the protocol never failed, it must be the case that every time line 10 of BOUNDED EGT DYNAMICS is reached, the moves $(x^k)^T A y^k$ are played three times in succession. Furthermore, since the column player always sets $\tilde{y} := y^k$ in line 15 of the CHECK CONV protocol, it must be the case that, by deviating, the column player could have improved his payoff by no more than ϵ_k in each of the three rounds.
- Line 15 - This is a SAFEPLAYROW routine. Notice that, in line 8 of the SAFEPLAYROW routine, the column player ensures that each entry in the vector $|u^T - (1 - \delta_k)(x^k)^T A|$ has absolute value no more than $\delta_k |A|_{max}$. In particular, for all $j \in \{1, 2, \dots, m\}$, we have

$$\begin{aligned} |u^T - (x^k)^T A|_j &\leq |u^T - (1 - \delta_k)(x^k)^T A|_j \\ &\quad + |\delta_k (x^k)^T A|_j \\ &\leq 2\delta_k |A|_{max}. \end{aligned}$$

Therefore, we know that payoff of the column player's best response against the loss vector u^T he observes in this round differs from the payoff of the best response to the loss vector $(x^k)^T A$ (observed previously) by no more than $2\delta_k |A|_{max}$. Since the column player has already verified that y^k is within ϵ_k of a best response to $(x^k)^T$, we conclude that by deviating in this round the column player could have improved his payoff by at most $2\delta_k |A|_{max} + \epsilon_k$.

- Line 17- In this line, the players perform a SAFEPLAYCOL routine. In this routine, the column player played the strategy $\delta_k y + (1 - \delta_k)y^k$. We know that the payoff of playing $\delta_k y + (1 - \delta_k)y^k$ against any move x is within $2\delta_k |A|_{max}$ of playing y^k against x . Since the payoff of y^k is within ϵ_k of the maximum possible payoff against x^k , we conclude that the payoff received by the column player in this round is within $2\delta_k |A|_{max} + \epsilon_k$ of his best response to the opponent's move.
- Line 27- The analysis of this round is identical to the analysis of line 17. Therefore, we conclude that by deviating the column player could have improved his payoff in this round by no more than $2\delta_k |A|_{max} + \epsilon_k$.
- Line 29- The analysis is identical to line 15. By deviating, the column player could have improved his payoff by no more than $2\delta_k |A|_{max} + \epsilon_k$.

This analysis gives us Lemma 4.1, which we formally prove below. The lemma, combined with the fact that the BOUNDED EGT DYNAMICS(b) protocol instructs

a player to use the multiplicative weights update algorithm if he ever declares a YIELD or QUIT failure, implies the following corollary:

COROLLARY C.1. *For any fixed value of b , the protocol BOUNDED EGT DYNAMICS(b) is no-regret. Furthermore, if $b \geq \sqrt{\ln n \ln m}$ and if both players honestly follow the protocol, then the average payoff received in the first T rounds of the protocol will differ from the Nash equilibrium payoff by at most*

$$O\left(\frac{|A|_{max} \sqrt{\ln n \ln m} \ln(T+3)}{T}\right).$$

Proof. [Proof of Lemma 4.1]

We obviously have the inequality

$$\max_{y \in \Delta_m} \sum_{t=1}^T x_t^T A y \leq \sum_{t=1}^T \max_{y \in \Delta_m} x_t^T A y.$$

We now upper bound the right hand side of the above expression.¹⁶ From above, we know that

$$\max_{y \in \Delta_m} x_1^T A y \leq x_1^T A y_1 + 2|A|_{max}.$$

In all later rounds, we have

$$\max_{y \in \Delta_m} x_t^T A y \leq x_t^T A y_t + 2\delta_k |A|_{max} + \epsilon_k,$$

where $k = \lfloor \frac{t-2}{5} \rfloor$. (Note that there are 5 rounds played for each value of k .) Therefore, we can bound

$$\begin{aligned} \sum_{t=1}^T \max_{y \in \Delta_m} x_t^T A y &\leq \sum_{t=1}^T x_t^T A y + 5 \sum_{k=0}^{\lfloor \frac{T-2}{5} \rfloor} (2\delta_k |A|_{max} + \epsilon_k) \\ &= \sum_{t=1}^T x_t^T A y + 10|A|_{max} \sum_{k=0}^{\lfloor \frac{T-2}{5} \rfloor} \frac{1}{(k+1)^2} \\ &\quad + 5 \sum_{k=0}^{\lfloor \frac{T-2}{5} \rfloor} \frac{4|A|_{max} b}{k+1}. \end{aligned}$$

We now use the bound

$$\begin{aligned} 10|A|_{max} \sum_{k=0}^{\lfloor \frac{T-2}{5} \rfloor} \frac{1}{(k+1)^2} &\leq 10|A|_{max} \sum_{k=0}^{\infty} \frac{1}{(k+1)^2} \\ &= \frac{10|A|_{max} \pi^2}{6} < 17|A|_{max} \end{aligned}$$

¹⁶In general, $\max_y \sum x_t^T A y$ may be significantly smaller than $\sum \max_y x_t^T A y$. In this particular case, however, the expressions will be very close to each other. The reason is that, since no failures have been detected, the x_t will be very close to Nash equilibrium strategies.

and the bound

$$5 \sum_{k=0}^{\lfloor \frac{T-2}{5} \rfloor} \frac{4|A|_{\max} b}{k+1} = 20|A|_{\max} b \sum_{s=1}^{\lfloor \frac{T-2}{5} \rfloor + 1} \frac{1}{s} \leq 20|A|_{\max} b (1 + \ln(\frac{T+3}{5})).$$

Note that

$$1 + \ln((T+3)/5) = \ln e + \ln((T+3)/5) \leq \ln(T+3).$$

The result of the lemma now follows.

D The NoRegretEgt Protocol

We present the NOREGRETEGT protocol.

```

1: function NOREGRETEGT
2:   Run INITIALIZATION
3:    $R, C$ :  $b := 1, k := 0$ 
4:
5:   while no QUIT errors have occurred do
6:     Run BOUNDEEGTDYNAMICS( $b$ ), starting
       from line 7 of that protocol, using the most recent
       values of  $k, x^k, y^k$ . Continue until a YIELD( $s$ ) fail-
       ure occurs
7:      $R, C$ : Run an additional  $(\max(2b, s))^4$ 
       rounds of the MWU algorithm.
8:      $R, C$ : Set  $b := \max(2b, s)$ .
9:   end while
10:   $R, C$ : Run the MWU algorithm forever
11: end function

```

D.1 Bounding the Regret. Let us look at some execution of the NOREGRETEGT algorithm where the column player plays honestly (we make no assumptions about the row player at the moment), and suppose that T total rounds of the game have been played thus far. We will now formally bound the column player's total regret. (His total regret is the difference between the payoff of his optimal single strategy against the opponent's history and his payoff actually received.) We can write $T = T_{EGT} + T_{MW}$, where T_{EGT} is the number of rounds of the game which have been played when the column player was in line 6 of the NOREGRETEGT protocol, and T_{MW} is the number of rounds which have been played during lines 7 or 10 of the protocol.

Let b be the largest (most recent) value which has been used as the input to BOUNDEEGTDYNAMICS(b) on line 6 of NOREGRETEGT. Notice that, if we ignore the rounds of the game which occurred closely before YIELD failures, the the remaining rounds from line 6

constitute a failure-free execution of BOUNDEEGTDYNAMICS(b).¹⁷

There have been at most $\log_2 b$ total YIELD failures thus far, since we at least double the value of b in line 8 of NOREGRETEGT. Since we restart from line 7 of the BOUNDEEGTDYNAMICS protocol every time there is a YIELD failure (regardless of the particular line of BOUNDEEGTDYNAMICS on which the failure occurred), it is possible that at most the 5 rounds prior to the failure will be "redone" when we restart after the failure.¹⁸ For simplicity, we will (unnecessarily loosely) upper bound the regret during each of these "redone" rounds as $2|A|_{\max}$. Let the total number of "redone" rounds be T_{redone} .

From Lemma 4.1, we can upper bound the column player's total regret during the T_{EGT} rounds from line 6 by

$$19|A|_{\max} + 20|A|_{\max} b \ln((T_{EGT} - T_{redone}) + 3) + 2|A|_{\max} T_{redone}.$$

Since $T_{redone} \leq 5 \log_2 b$ (since at most 5 rounds are "redone" after a YIELD failure), we can upper bound this total regret by

$$19|A|_{\max} + 20|A|_{\max} b \ln(T_{EGT} + 3) + 2|A|_{\max} \log_2 b.$$

During the T_{MW} rounds of the game for which the column player was on lines 7 or 10 of NOREGRETEGT, we can upper bound the total regret using Lemma 1.1 by

$$\frac{2|A|_{\max} \sqrt{T_{MW} \ln m}}{\sqrt{2} - 1} \leq 5|A|_{\max} \sqrt{T_{MW} \ln m}.$$

Therefore, the column player's average regret over the T rounds is upper bounded by

$$\frac{1}{T} \left(19|A|_{\max} + 20|A|_{\max} b \ln(T_{EGT} + 3) + 2|A|_{\max} \log_2 b + 5|A|_{\max} \sqrt{T_{MW} \ln m} \right)$$

¹⁷The key point is that these rounds constitute a failure-free execution of BOUNDEEGTDYNAMICS(b) even if, when they were played, the "presumed upper bound" input to BOUNDEEGTDYNAMICS was something other than b . This is because the value of b only impacts the execution of BOUNDEEGTDYNAMICS in the case that a YIELD error occurs.

¹⁸Since we restart from line 7 of BOUNDEEGTDYNAMICS, it is possible that we will redo at most 5 rounds of the game after we readjust the b value. Also, note that, for example, the row player's move on line 8 of the CHECKCONV routine differs depending on whether or not a YIELD failure will be declared after the next round. This is one of the lines which will be "redone."

and hence is upper bounded by

$$\frac{1}{T} \left(19|A|_{\max} + 20|A|_{\max} b \ln(T+3) + 2|A|_{\max} \log_2 b + 5|A|_{\max} \sqrt{T \ln m} \right).$$

The key observation is that, because of the relation on line 8, we will always have $b \leq \sqrt[4]{T}$. Therefore, we can upper bound the average regret by

$$\begin{aligned} & \frac{19|A|_{\max}}{T} + \frac{20|A|_{\max} \sqrt[4]{T} \ln(T+3)}{T} \\ & + \frac{2|A|_{\max} \log_2(\sqrt[4]{T})}{T} + \frac{5|A|_{\max} \sqrt{\ln m}}{\sqrt{T}} \\ & \leq \frac{19|A|_{\max}}{T} + \frac{20|A|_{\max} \sqrt[4]{T} \ln(T+3)}{T} + \frac{|A|_{\max} \ln(T)}{T} \\ & + \frac{5|A|_{\max} \sqrt{\ln m}}{\sqrt{T}}. \end{aligned}$$

We can use a nearly identical argument to upper bound the row player's average regret in the case that the row player is honest, regardless of the actions of the column player.

This yields Theorem 4.1.

D.2 Convergence with Honest Players. We now consider an execution of the NOREGRETEGT protocol in which both the row and column players honestly follow the prescribed protocol. The key observation is that once b becomes greater than $\sqrt{\ln n \ln m}$, there will never be any YIELD failures. Therefore, the total number of YIELD failures will be at most $\log_2(2\sqrt{\ln n \ln m})$.

Furthermore, the total number of rounds with the players in the MWU phase (line 7 of NOREGRETEGT) is at most

$$\begin{aligned} \sum_{l=1}^{\log_2(2\sqrt{\ln n \ln m})} (2^l)^4 & \leq 2(2\sqrt{\ln n \ln m})^4 \\ & = 32(\ln n)^2(\ln m)^2. \end{aligned}$$

Furthermore, in this honest execution of the NOREGRETEGT protocol, at most $5 \log_2(2\sqrt{\ln n \ln m})$ rounds of BOUNDEEGTDYNAMICS will be “redone” following YIELD errors (see Section D.1). Therefore, using Lemmas 1.1 and 4.1, (and bounding the regret by $2|A|_{\max}$ during each “redone” round) we can upper bound the column player's total regret over T rounds by

$$\begin{aligned} & 10|A|_{\max} \log_2(2\sqrt{\ln n \ln m}) \\ & + 5|A|_{\max} \sqrt{32(\ln n)^2(\ln m)^2 \ln m} + \\ & + 19|A|_{\max} + 20|A|_{\max} \sqrt{\ln n \ln m} \ln(T+3). \end{aligned}$$

This yields the final theorem of the paper, Theorem 4.2.